

Benvenuti a tutti. In questa presentazione verrà illustrato l'implementazione di un Covert Channel che, tramite il protocollo ICMP, permetterà la trasmissione di dati fra due macchine. Inizialmente introdurremo il protocollo ICMP e cosa sono i Covert Channel. Successivamente l'implementazione del canale di comunicazione. Per finire con i risultati relativi ai test eseguiti e con alcune osservazioni.

ICMP è un protocollo che opera al livello di rete e risulta fondamentale per la diagnostica delle reti. Per questo, non può essere completamente disabilitato.

Un Covert Channel è una canale che permette il trasferimento di dati in maniera non autorizzata. Solitamente sfrutta vulnerabilità o comportamenti non previsti (nei sistemi) per non generare segnali di un uso improprio di quest'ultimi.

Un Covert Channel viene determinato da alcune caratteristiche.

- **Furtività:** La capacità di evitare di attirare attenzioni su di se.
- **Capacità di trasmissione:** La quantità di dati che il canale trasmette.
- **Indistinguibilità:** Durante la trasmissione il funzionamento delle risorse utilizzate rimane inalterato.

In base a come si inviano i dati, si dividono in:

- **Timing Covert Channel:** in cui le informazioni vengono inviate attraverso il tempo
- **Storage Covert Channel:** l'informazione viene inserita in un'area di memoria condivisa ed accessibile dalle entità coinvolte.
- **Behavioural Covert Channel:** le informazioni vengono codificate mediante gli eventi che avvengono nel sistema.

Ora passiamo ad illustrare lo sviluppo della comunicazione fra le entità. Per una prima implementazione si è analizzato il progetto **icmp tunnel** per degli spunti. Da ciò si è definita la presenza di un'entità che farà da proxy fra le due macchine che d'ora in poi chiameremo attaccante e vittima.

Le fasi che avvengono durante la comunicazione sono le seguenti:

Nella fase di inizializzazione:

l'**attaccante** definisce

- l'indirizzo IP della vittima
- la lista dei proxy che si userà per la comunicazione con la vittima
- il Covert Channel per l'invio di dati

il **proxy** invece definisce:

- l'indirizzo IP dell'attaccante. Servirà successivamente durante la fase di definizione delle connessioni.

la **vittima** infine definisce:

- la quantità di connessioni attive necessarie per l'invio dei dati.

Le entità procederanno poi a stabilire le connessioni fra di loro.

L'**attaccante** inizia ricavando quali proxy sono disponibili. Ovvero quelli con cui può comunicare e che a loro volta comunicano con la vittima. E lo fa in questo modo:

- si connette ad un proxy ed aspetta che la connessione venga confermata
- Successivamente aspetta l'aggiornamento da proxy; che indicherà se quest'ultimo comunica con la vittima.
- dal risultato l'attaccante decide poi se scartare il proxy.

La procedura termina una volta che tutti proxy specificati inizialmente aggiornano l'attaccante.

Il **proxy** invece :

- inizialmente rimane in attesa che l'attaccante si connetta e che gli indichi l'indirizzo della vittima ed il Covert Channel utilizzato.
- successivamente tenterà di creare una comunicazione con la vittima. Aggiungerà poi l'attaccante sul risultato

La **vittima** rimarrà in attesa finché non raggiunge il numero minimo di connessioni stabilite. Durante la connessione, l'entità che si vuole connettere indicherà il Covert Channel che vorrà utilizzare per l'invio dei dati.

Una volta che tutte le entità sono connesse fra di loro; l'attaccante inizierà con l'invio delle informazioni. Quindi:

- inserisce nel terminale un comando od un messaggio
- sceglie uno dei proxy disponibile che inoltrerà l'informazione (ai proxy restanti indicherà di aspettare i dati dalla vittima)
- successivamente aspetterà che i proxy finiscano di inoltrargli le informazioni ricevute dalla vittima.

Il proxy così come la vittima rimangono in attesa di un comando o di un messaggio.

Il primo quando lo riceve lo inoltra alla vittima e poi aspetta i dati dalla vittima; che verranno poi inoltrati all'attaccante.

La vittima invece:

- procede solamente a confermare la ricezione, se il dato ricevuto è un semplice messaggio.

- se invece si tratta di un comando, procede ad eseguirlo e poi invierà all'attaccante i dati, sfruttando tutte le connessioni disponibili.

Una volta ricevuti tutti i dati dai proxy, l'attaccante li riordinerà per ricavare il messaggio correttamente. Successivamente deciderà se inviare un ulteriore comando o se terminare la comunicazione.

1. TIMING COVERT CHANNEL

Procediamo ora ad illustrare l'implementazione delle varie tipologie di Covert Channel. In un Timing Channel, per poter temporizzare i dati da inviare c'è bisogno di una funzione che calcola l'intervallo temporale associato al dato ed il viceversa. Ovvero dato un intervallo temporale si calcola il relativo dato.

IMPLEMENTAZIONE DELLA FUNZIONE

In una prima codifica si ricava il delay basato sulle distanze che i dati devono avere fra di loro.

Si avrà quindi un tempo base, assegnato al primo dato, ed i successivi avranno una distanza fissa dal dato precedente. Il vantaggio è una maggiore tolleranza ai ritardi di rete; tuttavia lo svantaggio sono i tempi che si aspettano.

Per risolvere il problema si è pensato di ridurre i valori inviabili. In particolare si invieranno solo i caratteri stampabili; che nella codifica ASCII vanno da 32 a 126. Tuttavia questo ci lega ad una codifica specifica e inoltre porta alla perdita di informazioni se mai si mandassero file non contenenti testo.

Prendendo quindi spunto da ICMP Exfill si sviluppa meglio l'idea. Il tempo di delay ora viene indicato dal valore intero del byte da inviare. Dopodiché si normalizzerà il risultato fra un valore minimo ed uno massimo. In questo modo il range dei delay verrà ristretto ad un intervallo definito che permetterà di regolare la velocità di trasmissione, sacrificando però la sensibilità della comunicazione ai ritardi di rete.

Successivamente, alla codifica precedente, si è aggiunto del rumore. Questo per avere uno schema dei tempi meno prevedibile; con la conseguenza che verrà rilevato con maggiore difficoltà. Siccome ora il delay non è più costante ma varierà fra un range di valori; ciò necessita che, per potersi scambiare i dati correttamente, il mittente e il destinatario devono sincronizzarsi sulla quantità di rumore aggiunto.

- Il mittente crea un valore randomico per il rumore e le entità si sincronizzeranno sul seed usato e sul numero di estrazioni effettuate.
- Il mittente indica il rumore generato nel pacchetto stesso ed il destinatario otterrà il valore dalla risorsa condivisa (il pacchetto).

2. STORAGE COVERT CHANNEL

In uno Storage Covert Channel la risorsa condivisa sarà il pacchetto ICMP inviato ed il mittente inserirà i dati nei campi del messaggio. Le tabelle indicano i campi utilizzati nei messaggi oltre al numero di byte inseribili. Dei campi particolari sono Header+64 bit, Invoking Packet, Unused e Timestamp. Siccome il dato non può essere inserito direttamente.

1. CAMPO HEADER+64 BIT / INVOKING PACKET

Nel campo è presente l'intestazione IP più 64 bit del datagram sul quale è venuto un errore che nel nostro caso rappresenterà un pacchetto ICMP.

In cui, nell'intestazione IPv4 i dati saranno inseriti nei campi:

- Time to live
- Total length
- Identification

Mentre nell'intestazione ICMPv4 i dati saranno inseriti nei campi:

- Identifier
- Sequence

Come per Header+64 bit, anche per Invoking Packet si indicherà un messaggio IP/ICMP. Nel nostro caso si userà l'intestazione IPv4 che è di 40 byte, ed il protocollo ICMPv4, la cui intestazione è di 8 byte.

Quindi nell'intestazione IPv6 i dati saranno inseriti nei campi:

- Payload Length
- Next Header
- Hop Limit

Mentre nell'intestazione ICMPv6 i dati saranno inseriti nei campi:

- Identifier
- Sequence

2. CAMPO UNUSED

Dalle specifiche RFC (792 e 4434) il campo deve essere vuoto. Per questo si è usata la libreria struct per la costruzione del pacchetto (avente i dati all'interno del campo) e successivamente il framework Scapy per l'invio.

3. CAMPO TIMESTAMP

Contiene i millisecondi passati dalla mezzanotte UT. Siccome il dato indica una data temporale il byte è stato inserito nella parte dei millisecondi. Nelle altre parti della data non è stato possibile

siccome rappresentano le ore, i minuti ed i secondi. L'alternativa poteva essere di inserire il dato per l'intera dimensione del campo ma avrebbe portato a tempi non validi.

3. BEHAVIOURAL COVERT CHANNEL

Nel Behavioural Covert Channel invece, l'evento è definito dall'arrivo di un messaggio ICMP. Quindi ad una determinata coppia (Tipologia, Codice) verrà associato un dato. Con questa implementazione si riescono a codificare 4 bit alla volta, siccome il numero di eventi possibili sono 17. L'evento che rimane inutilizzato, è stato usato per indicare l'inizio e la terminazione dell'invio dei dati.

Un problema dell'implementazione, sono le tipologie deprecate (Information Request e Source Quench). Queste tipologie di messaggi possono comunque essere inviate; tuttavia non è assicurata la ricezione siccome le linee guida indicano ai router di ignorarli e all'host utente di non mandarli.

TEST E RISULTATI

Inizialmente si sono effettuati test in cui si inviava il dato una singola volta. Ma siccome non si è avuto risultati utilizzabili; uno step successivo è l'invio degli stessi dati più volte (in particolare tre). Inoltre si sono definiti dei parametri per verificare meglio cosa avviene:

- La quantità di dati inviati (1KB, 10KB, 100KB, 1MB).
- I campi usati dal Covert Channel. Per esempio nei messaggi Icmp Echo le variazioni si baseranno sull'utilizzo o meno del campo Identifier o del campo Data.
- Il delay fra i blocchi di dati. Siccome i dati verranno suddivisi in blocchi da 100KB; si può decidere se avere un invio sequenziale dei blocchi oppure se avere un delay fra un blocco ed il successivo. La divisione in blocchi permette l'invio di file con grosse dimensioni evitando di congestionare il canale di comunicazione.
- Tempo di riposo prima di un nuovo invio (per esempio 1 ora, 30 minuti, 15 minuti).
- Il mittente del messaggio. Nel messaggio si può inserire lo stesso mittente oppure inserire per ogni blocco mittenti diversi. Nel caso gli indirizzi verranno presi da un pool indicando o gli indirizzi IP attualmente assegnati a degli host nella rete oppure gli indirizzi IP non associati ad alcun dispositivo.

Dai risultati si ricava che l'uso di diversi mittenti e tempi di attesa abbastanza lunghi riducono la gravità assegnatagli.

Inoltre nei casi in cui si inviavano sequenzialmente i dati; nei test non sono mai stati identificati. Tuttavia una grande quantità di dati congestionerebbe il canale di comunicazione, con la conseguente creazione di rumore. Che attirerà l'attenzione dei sistemi di sicurezza.

CONCLUSIONI SULL'IMPLEMENTAZIONE DEI COVERT CHANNEL

In conclusione si è visto come il protocollo ICMP, fondamentale per la diagnostica delle reti, può essere utilizzato per la creazione di un canale di comunicazione nascosto. E che dai test effettuati, modificando il mittente del pacchetto e impostando i giusti tempi di attesa, si è riusciti a diminuire la gravità del canale.

Quindi siccome date le sue funzionalità, il protocollo non può essere completamente disabilitato; si devono effettuare delle accortezze:

- permettere, nel traffico di rete, solo le tipologie di messaggi essenziali.

Fra queste bisognerà monitorare il loro utilizzo e il contenuto:

- si dovranno prevedere metodi per l'ispezione dei pacchetti .
- bisogna limitare la quantità di messaggi che un host può inviare al secondo
- aggiungere del ritardo non costante ai messaggi prima di inoltrarli; così da impedire eventuali comunicazioni sui tempi.

Infine per vedere se il mittente sta inviando i dati secondo uno schema anomalo si possono effettuare delle analisi statistiche rispetto ad un flusso di rete nelle condizioni normali.